

Day 4, Session 1: Distributed Task Queues & Scientific Computing (Celery & SCOOP)

Duration: 2 hours (theory + live demonstrations)

Prerequisites: Days 1, 2, & 3 (Multiprocessing, Networking, MPI)

Demos folder: [day4/demo/](#)

Table of Contents

1. [Beyond Single-Machine Concurrency](#)
 2. [Distributed Task Queues with Celery](#)
 - [The 4-Component Architecture](#)
 - [Defining and Registering Tasks](#)
 - [Asynchronous Execution: `delay\(\)` and `apply\_async\(\)`](#)
 - [Inspecting Task State & AsyncResult](#)
 - [Complex Workflows: Chains and Groups](#)
 3. [Scientific Computing with SCOOP](#)
 - [What is SCOOP?](#)
 - [SCOOP Architecture: ZeroMQ & Scalability](#)
 - [Distributed `map\(\)` over Heterogeneous Nodes](#)
 - [Comparing `multiprocessing.Pool` vs SCOOP](#)
 4. [Session 1 Summary](#)
-

1. Beyond Single-Machine Concurrency

On Day 1 & 2, we worked on shared-memory parallelism using multiple cores inside a single server. On Day 3, we explored MPI for tightly coupled supercomputing clusters.

However, many real-world enterprise and scientific workflows are **asynchronous, distributed, and loosely coupled**:

- Processing millions of image uploads or video encoding jobs across cloud virtual machines.
- Running large genetic algorithms or evolutionary simulations across workstations in a laboratory.
- Decoupling user-facing web services from long-running background data pipelines.

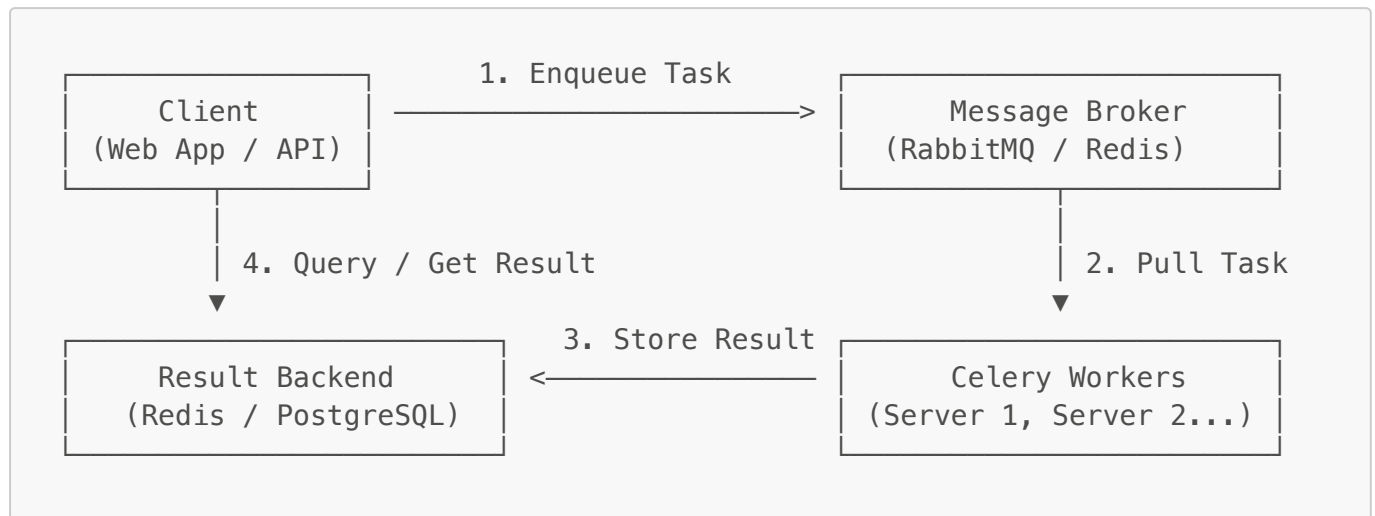
This is the domain of **Distributed Task Queues** and **Distributed Scientific Python frameworks**.

2. Distributed Task Queues with Celery

Celery is an open-source asynchronous distributed task queue/job queue based on distributed message passing. It allows Python programs to offload tasks to a farm of worker processes spread across multiple servers.

The 4-Component Architecture

Celery consists of four primary building blocks:



1. **Client (Producer)**: The Python code requesting that a task be executed.
2. **Message Broker**: A high-performance intermediary (usually **RabbitMQ** or **Redis**) that stores messages and routes them to available workers.
3. **Workers (Consumers)**: Dedicated background processes running on one or more servers that consume tasks from the queue and compute them.
4. **Result Backend**: Stores the return values and execution states (PENDING, STARTED, SUCCESS, FAILURE) of tasks.

Defining and Registering Tasks

You define Celery tasks by decorating standard Python functions with `@app.task`:

```

from celery import Celery

# Create Celery application instance
app = Celery(
    'my_tasks',
    broker='redis://localhost:6379/0',
    backend='redis://localhost:6379/1'
)

@app.task
def process_data(dataset_id, factor):
    """A task that can be distributed across any number of worker
    machines."""
    print(f"Processing dataset {dataset_id}...")
    return dataset_id * factor
  
```

Asynchronous Execution: `delay()` and `apply_async()`

When you call a task using `.delay()`, it **does not** execute locally. Instead, Celery serializes the function call into a JSON message, pushes it to the broker, and returns immediately with an **AsyncResult** handle:

```
# Calling standard function (blocks locally):
# result = process_data(101, 2.5)

# Calling distributed Celery task (non-blocking):
async_res = process_data.delay(101, 2.5)

print("Task dispatched! Task ID:", async_res.id)
print("Client continues running without waiting...")
```

`.apply_async(args, kwargs, ...)`

For advanced requirements, `apply_async()` offers fine-grained controls:

- **Countdown / ETA:** Delay execution by N seconds or schedule for a specific time (`countdown=60`).
- **Retries:** Automatically retry on connection failures.
- **Routing:** Send high-priority tasks to a dedicated high-memory queue.

Inspecting Task State & AsyncResult

The **AsyncResult** object allows the client to poll or await the computation:

Method / Property	Description
<code>async_res.ready()</code>	Returns True if the worker has finished the task
<code>async_res.successful()</code>	Returns True if task completed without raising an exception
<code>async_res.status</code>	Current lifecycle state: PENDING -> STARTED -> SUCCESS / FAILURE
<code>async_res.get(timeout=...)</code>	Blocks until the worker stores the result in the backend

```
# Check status without blocking
if async_res.ready():
    print("Result is ready:", async_res.get())
else:
    print(f"Task is still {async_res.status}...")
```

Complex Workflows: Chains and Groups

Celery includes powerful workflow primitives called **Canvas**:

1. Chains (Sequential Pipeline)

```
from celery import chain

# Pipeline: fetch -> transform -> save
workflow = chain(fetch_data.s(url) | process_data.s() | save_to_db.s())
workflow.apply_async()
```

Executes a list of tasks in parallel across all available workers in the cluster:

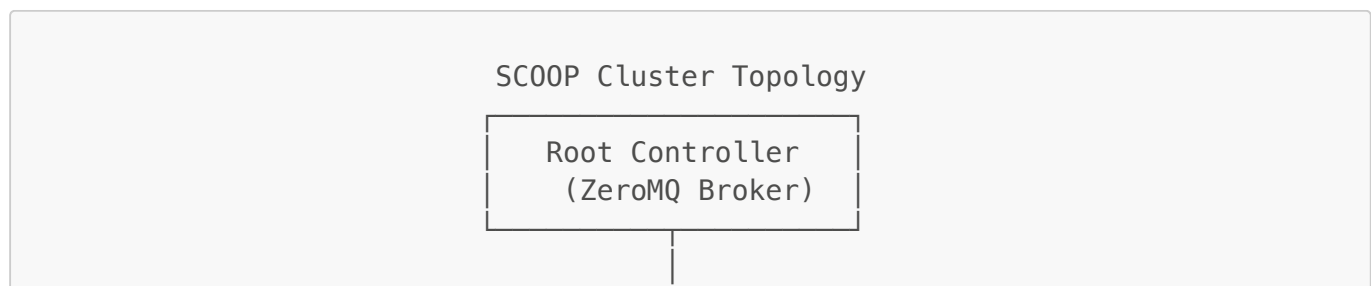
```
from celery import group

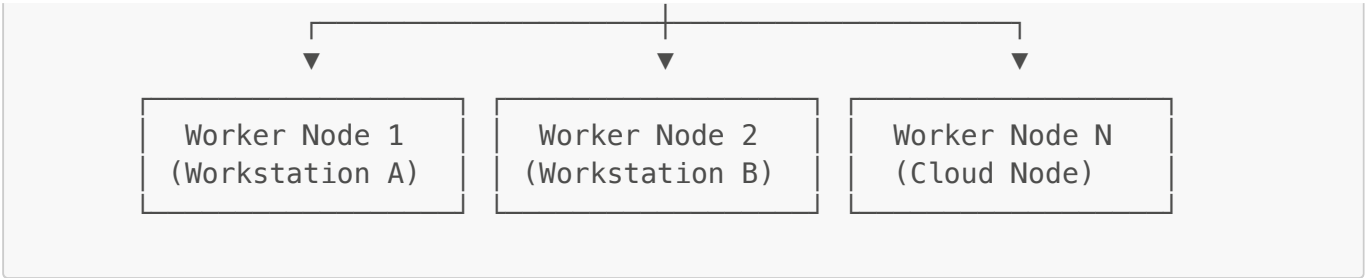
# Distribute 100 images across the worker cluster in parallel:
job = group(resize_image.s(img_id) for img_id in image_ids)
result_group = job.apply_async()
```

- `01_celery_distributed_tasks.py` —
 1. `demo_celery_task_definition()` — Asynchronous `.delay()`, non-blocking submission, and `AsyncResult`
 2. `demo_celery_workflows()` — Chaining and grouping tasks across workers

What is SCOOP?

- Evolutionary / genetic algorithms (e.g. DEAP library integration).
- Monte Carlo simulations.
- Hyperparameter search in machine learning.





SCOOP Architecture: ZeroMQ & Scalability

Unlike Celery (which requires installing external services like Redis or RabbitMQ), SCOOP communicates directly using **ZeroMQ** sockets:

- No central database or broker installation required.
- Automatically handles load balancing and task stealing between fast and slow nodes.
- Integrates directly with cluster resource managers like **SLURM**, **PBS**, and **Torque**.

Distributed `map()` over Heterogeneous Nodes

SCOOP adheres closely to Python's native `concurrent.futures` API:

```
from scoop import futures

def evaluate_simulation(param):
    # Heavy scientific simulation
    return run_physics_sim(param)

if __name__ == '__main__':
    parameters = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

    # Transparently distributed across all cores and all machines in the
    # hostfile!
    results = list(futures.map(evaluate_simulation, parameters))
    print("Simulation results:", results)
```

To run across multiple physical machines over SSH:

```
python3 -m scoop --hostfile hosts.txt my_simulation.py
```

Comparing `multiprocessing.Pool` vs SCOOP

Feature	<code>multiprocessing.Pool</code>	SCOOP
Cluster Scaling	Single machine only	Multi-node network cluster
Interconnect	OS Pipes / Shared Memory	High-performance ZeroMQ sockets

Feature	<code>multiprocessing.Pool</code>	<code>SCOOP</code>
Dynamic Load Balancing	Static chunking	Work stealing (fast nodes pull more work)
Cluster Schedulers	None	Native support for SLURM, GridEngine
API	<code>pool.map(func, data)</code>	<code>futures.map(func, data)</code>

Demo Time

Now, let's see these theoretical topics in action.

- `02_scoop_scientific_map.py` —
 1. `demo_scoop_map()` — Distributed Monte Carlo calculation using `scoop.futures.map()`
 2. `demo_scoop_architecture()` — Architectural differences between local multiprocessing and cluster SCOOP

Session 1 Summary

Technology	Best Suited For	Broker / Transport	Key Strengths
Celery	Web backends, ETL jobs, distributed tasks	Redis / RabbitMQ	Durable queues, retries, task workflows (chains/groups)
SCOOP	Scientific computing, optimization	ZeroMQ (peer-to-peer)	Easy setup, no database broker required, SLURM integration

Break time! Next up is Session 2: Remote Objects & RPC (Pyro4, RPyC, PyCSP) → [README-session2.md](#)